MIC regression

Lucy Harrison

2026-03-17

Preamble

Fit regression models

Complete MIC data (NOS)

```
# apply log to the mics (mix of NOS and 90):
dat_nos_only <- dat_nos_only %>%
  mutate(mic_logged = log2(micnos_1))

# fit a model weighted by inverse sqrt sample sizes to determine residual errors:
# weights in full model
mod_init <- lm(mic_logged ~ isolate_yr, dat_nos_only, weights = 1/sqrt(count))
summary(mod_init)
```

```
##
## Call:
## lm(formula = mic_logged ~ isolate_yr, data = dat_nos_only, weights = 1/sqrt(count))
##
## Weighted Residuals:
##      Min       1Q   Median       3Q      Max
## -2.98965 -0.39038 -0.00134  0.01035  2.45339
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept) -5.2795876 13.0483883  -0.405    0.686
## isolate_yr  -0.0003462  0.0065127  -0.053    0.958
##
## Residual standard error: 0.6195 on 170 degrees of freedom
## Multiple R-squared:  1.662e-05,  Adjusted R-squared:  -0.005866
## F-statistic: 0.002826 on 1 and 170 DF,  p-value: 0.9577
```

```
# setting weight to median absolute residuals
wts <- data.frame(isolate_yr = dat_nos_only$isolate_yr,
                  resid = mod_init$residuals) %>%
  group_by(isolate_yr) %>%
  summarise(wt = median(abs(resid)))

dat_nos_only <- dat_nos_only %>%
  left_join(wts)
```

```
## Joining with `by = join_by(isolate_yr)`
```

```r
mod_weighted_nos_only <- lm(mic_logged ~ isolate_yr, dat_nos_only, weights = wt)
confint(mod_weighted_nos_only)
```

```
##                    2.5 %     97.5 %
## (Intercept) -53.81826266 0.17663911
## isolate_yr   -0.00315681 0.02384291
```

```r
summary(mod_weighted_nos_only)
```

```
##
## Call:
## lm(formula = mic_logged ~ isolate_yr, data = dat_nos_only, weights = wt)
##
## Weighted Residuals:
##      Min       1Q   Median       3Q      Max
## -2.87548 -0.58970  0.01371  0.12923  2.96707
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept) -26.820812  13.676404  -1.961   0.0515 .
## isolate_yr    0.010343   0.006839   1.512   0.1323
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.8733 on 170 degrees of freedom
## Multiple R-squared:  0.01328,    Adjusted R-squared:  0.007472
## F-statistic: 2.287 on 1 and 170 DF,  p-value: 0.1323
```

```r
# this is slope on log-scale:
coefficients(mod_weighted_nos_only)[2]
```

```
## isolate_yr
## 0.01034305
```

```r
2**(coefficients(mod_weighted_nos_only)[2])
```

```
## isolate_yr
##   1.007195
```

```r
2**confint(mod_weighted_nos_only)[2,]
```

```
##     2.5 %    97.5 %
## 0.9978143 1.0166640
```

```r
dat_nos_only %>%
  # clarify this is right:
  group_by(author, pub_yr) %>%
  summarise(n = n()) %>%
  as.data.frame()
```

Adding random effect on study

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by author and pub_yr.
## i Output is grouped by author.
## i Use `summarise(.groups = "drop_last")` to silence this message.
```

```
## i Use `summarise(.by = c(author, pub_yr))` for per-operation grouping
##   (`?dplyr::dplyr_by`) instead.
```

```
##           author pub_yr  n
## 1   Abd El Ghany   2015  3
## 2   Abd El-Ghany   2015  1
## 3         Amusin   2026  4
## 4          Arhin   2009  3
## 5        Benouda   2009  3
## 6         Canton   2002 33
## 7        Cellesi   1996  3
## 8          Cohen   1996  4
## 9        Feldman   1987  1
## 10        Friaes   2025  1
## 11        Fujita   1994  1
## 12         Haddy   1982  3
## 13         Ikeda   2021  3
## 14 Kanellopoulou   1995  1
## 15           Kim   1985  2
## 16   Kriebernegg   1998  1
## 17        Musser   2020 54
## 18      Nakajima   1979  1
## 19      Nakajima   1982  3
## 20      Nakajima   1983  4
## 21      Reynolds   2025  6
## 22        Sakata   2013  2
## 23     Smeesters   2010  5
## 24       Southon   2020  7
## 25      Soyletir   2016  7
## 26    Stillerman   1970  3
## 27    Stillerman   1973  4
## 28         Tappe   2011  1
## 29    Torumkuney   2016  3
## 30       Vannice   2020  2
## 31       Varaldo   1999  3
```

```
X_LOWER <- 1965

# note misspelling of Abd El[ |-]Ghany

library(lme4)
```

```
## Loading required package: Matrix
##
## Attaching package: 'Matrix'
##
## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack
```

```
dat_nos_only_uncounted <- dat_nos_only %>%
  mutate(study = factor(paste(author, pub_yr, sep = ","))) %>%
  uncount(count)

mod_fixed <- lm(mic_logged ~ isolate_yr, dat_nos_only_uncounted)
tmp <- 2**confint(mod_fixed)
```

```r
tmp[2, 1:2]
```

```
##     2.5 %    97.5 %
## 0.9966422 0.9981378
```

```r
# we don't have multiple observations in the same study across multiple years - do we gain
# much structure from random effect on study?
mod_mixed <- lmer(mic_logged ~ isolate_yr + (1|study), dat_nos_only_uncounted)
mod_mixed <- lmer(mic_logged ~ isolate_yr + (1|country), dat_nos_only_uncounted)

summary(mod_mixed)
```

```
## Linear mixed model fit by REML ['lmerMod']
## Formula: mic_logged ~ isolate_yr + (1 | country)
##    Data: dat_nos_only_uncounted
##
## REML criterion at convergence: 13895.2
##
## Scaled residuals:
##     Min      1Q  Median      3Q     Max
## -6.2612 -0.0213  0.1076  0.3098  6.1028
##
## Random effects:
##  Groups   Name        Variance Std.Dev.
##  country  (Intercept) 0.6583   0.8113
##  Residual             0.2518   0.5018
## Number of obs: 9401, groups:  country, 33
##
## Fixed effects:
##               Estimate Std. Error t value
## (Intercept) -1.483e+01  1.114e+00 -13.314
## isolate_yr   4.135e-03  5.523e-04   7.488
##
## Correlation of Fixed Effects:
##            (Intr)
## isolate_yr -0.992
```

```r
# this is what we want in the text:
tmp <- 2**confint(mod_mixed)
```

```
## Computing profile confidence intervals ...
```

```r
# ... and now we're slightly positive
tmp[4, 1:2]
```

```
##    2.5 %   97.5 %
## 1.002119 1.003624
```

```r
# dropping code for plot up here:
newdata <- data.frame(isolate_yr = seq(X_LOWER, max(dat_nos_only$isolate_yr) + 2),
                      study = "NA,NA", country = "NA")

fixed_eff_pred <- data.frame(y = 2**predict(mod_fixed, newdata),
                             x = newdata$isolate_yr)

# bootstrap to get approximate CI
boot_ci <- bootMer(mod_mixed,
```

```
                  nsim = 100,
                  FUN = function(x){
                    predict(x, newdata = newdata, re.form = NA)
                    # ignoring random effect in prediction
                  })

mixed_eff_pred <- apply(boot_ci$t, 2, function(x){quantile(x, probs = c(0.025, 0.5, 0.975))}) %>%
  t() %>%
  as.data.frame() %>%
  setNames(nm = c("lwr", "fit", "upr")) %>%
  bind_cols(newdata %>% dplyr::select(isolate_yr))

YLIM <- range(c(dat_agg$mic_90_agg,
                dat_nos_only$micnos_1))
Y_LOWER <- log2(0.0018)
SIZE_LIMS <- range(c(dat_agg$count,
                     dat_nos_only$count))
yticks <- c(0, 0.002, 0.004, 0.008, 0.016, 0.032, 0.064, 0.12)

p1 <- ggplot(dat_nos_only %>%
             filter(!is.na(micnos_1)) %>%
             group_by(isolate_yr, micnos_1, who_name) %>%
             summarise(count = sum(count)) %>%
             mutate(who_name = factor(who_name, levels = c("Africa",
                                                           "Americas",
                                                           "Eastern Mediterranean",
                                                           "Europe",
                                                           "Southeast Asia",
                                                           "Western Pacific",
                                                           "Not assigned"))) %>%
         arrange(desc(count))) +
  geom_line(data = fixed_eff_pred, aes(x = x, y = y)) +
  geom_ribbon(data = mixed_eff_pred, aes(ymin = 2**lwr, ymax = 2**upr, x = isolate_yr),
              alpha = 0.5, fill = "grey", colour = NA) +
  geom_line(data = mixed_eff_pred, aes(x = isolate_yr, y = 2**fit),
            colour = "darkgrey", linewidth = 0.8) +
  geom_point(aes(x = isolate_yr, y = micnos_1,
                 col = who_name,
                 size = count), alpha = 0.5) +
  scale_color_discrete(name = "WHO Region", type = pal, drop = FALSE) +
  scale_size_continuous(range = c(1,10), name = "Number of isolates",
                        breaks = c(1, 10, 100, 1000), limits = SIZE_LIMS) +
  scale_y_continuous(breaks = yticks, minor_breaks = NULL, name = "MIC") +
  scale_x_continuous(limits = c(X_LOWER, 2022), expand = c(0,0),
                     breaks = seq(1970, 2010, 10), minor_breaks = NULL) +
  labs(x = "Year", ylab = "MIC") +
  coord_trans(y = "log2", ylim = YLIM) +
  theme_bw()
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by isolate_yr, micnos_1, and who_name.
## i Output is grouped by isolate_yr and micnos_1.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(isolate_yr, micnos_1, who_name))` for per-operation
```

```
##   grouping (`?dplyr::dplyr_by`) instead.
```

```
## Warning: `coord_trans()` was deprecated in ggplot2 4.0.0.
## i Please use `coord_transform()` instead.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

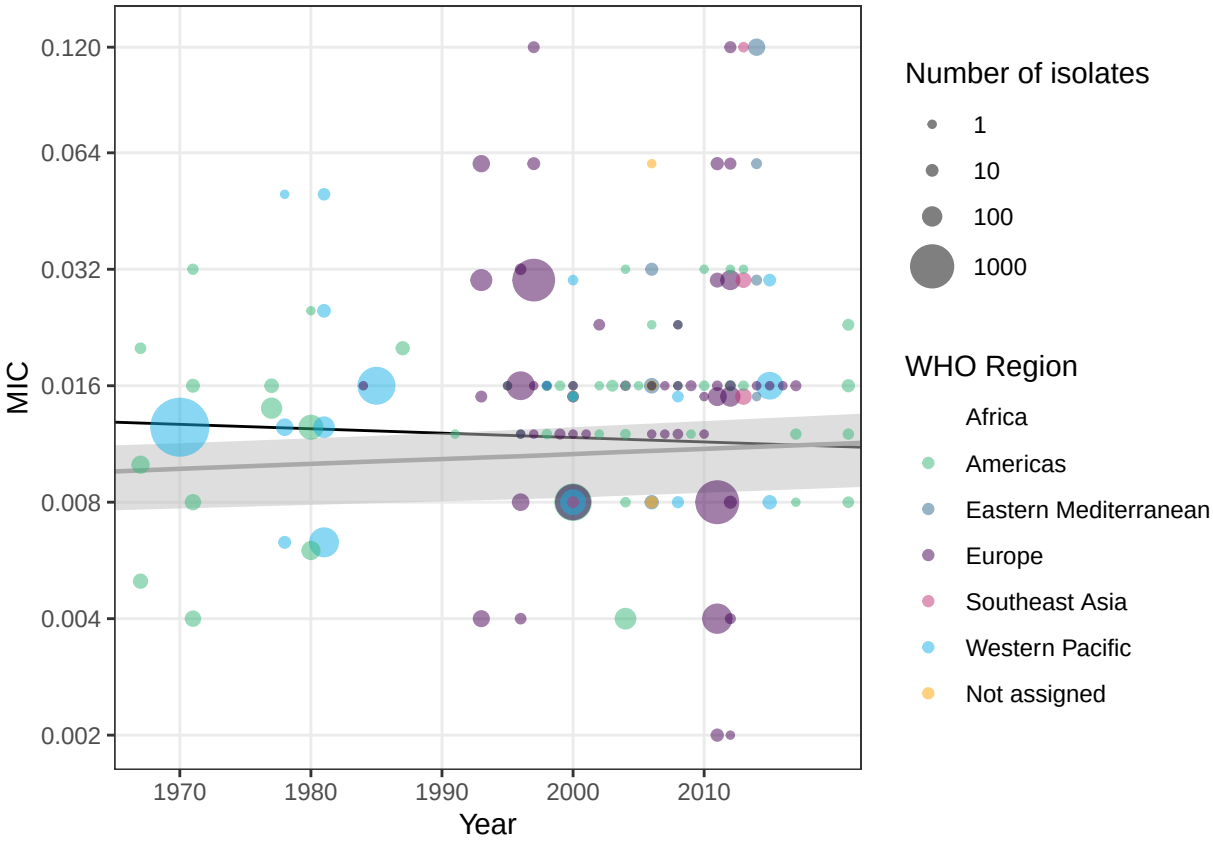
```
# okey well there's something weird happening here
p1
```

```
## Ignoring unknown labels:
## * ylab : "MIC"
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_line()`).
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_ribbon()`).
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_line()`).
```



All studies: MIC90

```
# apply log to the mics (mix of NOS and 90):
dat_agg <- dat_agg %>%
  mutate(mic_logged = log2(mic_90_agg))

# shouldn't these be one per study?
```

```r
dat_agg %>%
  filter(!is.na(mic_logged)) %>%
  group_by(author, pub_yr, country) %>%
  summarise(n = n()) %>%
  filter(n > 1)
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by author, pub_yr, and country.
## i Output is grouped by author and pub_yr.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(author, pub_yr, country))` for per-operation grouping
##   (`?dplyr::dplyr_by`) instead.
```

```
## # A tibble: 23 x 4
## # Groups:   author, pub_yr [9]
##    author     pub_yr country                          n
##    <chr>       <int> <chr>                        <int>
##  1 Bass         1994 Japan                            2
##  2 Buxbaum      2004 Austria                          4
##  3 Chazan       2015 Israel                           2
##  4 Coonan       1994 United States Of America (The)   2
##  5 Inoue        2006 Japan                            2
##  6 Macris       1998 United States Of America (The)   8
##  7 Marchese     2004 Italy                            2
##  8 d'Humières   2012 France                           2
##  9 <NA>           NA Australia                        2
## 10 <NA>           NA Austria                          2
## # i 13 more rows
```

```r
# fit a model weighted by inverse sqrt sample sizes to determine residual errors:
# weights in full model
mod_init = lm(mic_logged ~ isolate_yr, dat_agg, weights = 1/sqrt(count))
summary(mod_init)
```

```
##
## Call:
## lm(formula = mic_logged ~ isolate_yr, data = dat_agg, weights = 1/sqrt(count))
##
## Weighted Residuals:
##      Min       1Q   Median       3Q      Max
## -0.72980 -0.20952 -0.00096  0.11378  1.40323
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -1.943212   6.839889  -0.284    0.777
## isolate_yr  -0.002053   0.003418  -0.601    0.549
##
## Residual standard error: 0.3147 on 216 degrees of freedom
## Multiple R-squared:  0.001668,   Adjusted R-squared:  -0.002954
## F-statistic: 0.3608 on 1 and 216 DF,  p-value: 0.5487
```

```r
# setting weight to median absolute residuals
wts <- data.frame(isolate_yr = dat_agg$isolate_yr,
                  resid = mod_init$residuals) %>%
  group_by(isolate_yr) %>%
  summarise(wt = median(abs(resid)))
```

```r
dat_agg <- dat_agg %>%
  left_join(wts)
```

```
## Joining with `by = join_by(isolate_yr)`
```

```r
mod_weighted_agg <- lm(mic_logged ~ isolate_yr, dat_agg, weights = wt)
confint(mod_weighted_agg)
```

```
##                    2.5 %      97.5 %
## (Intercept) -28.82470320 19.03996449
## isolate_yr   -0.01250746  0.01143648
```

```r
summary(mod_weighted_agg)
```

```
##
## Call:
## lm(formula = mic_logged ~ isolate_yr, data = dat_agg, weights = wt)
##
## Weighted Residuals:
##     Min      1Q  Median      3Q     Max
## -2.7877 -0.3851 -0.0055  0.1692  3.4000
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept) -4.8923694 12.1421832  -0.403    0.687
## isolate_yr  -0.0005355  0.0060740  -0.088    0.930
##
## Residual standard error: 0.776 on 216 degrees of freedom
## Multiple R-squared:  3.598e-05,  Adjusted R-squared:  -0.004593
## F-statistic: 0.007772 on 1 and 216 DF,  p-value: 0.9298
```

```r
# this is slope on log-scale:
coefficients(mod_weighted_agg)[2]
```

```
##    isolate_yr
## -0.0005354911
```

```r
2**(coefficients(mod_weighted_agg)[2])
```

```
## isolate_yr
##  0.9996289
```

```r
2**confint(mod_weighted_agg)[2,]
```

```
##    2.5 %   97.5 %
## 0.991368 1.007959
```

Now for some plots

Common attributes:

```r
YLIM <- range(c(dat_agg$mic_90_agg, dat_nos_only$micnos_1))
Y_LOWER <- log2(0.0018)
SIZE_LIMS <- range(c(dat_agg$count, dat_nos_only$count))
yticks <- c(0, 0.002, 0.004, 0.008, 0.016, 0.032, 0.064, 0.12)
```

Complete MIC data (NOS only)

```
X_LOWER <- 1965

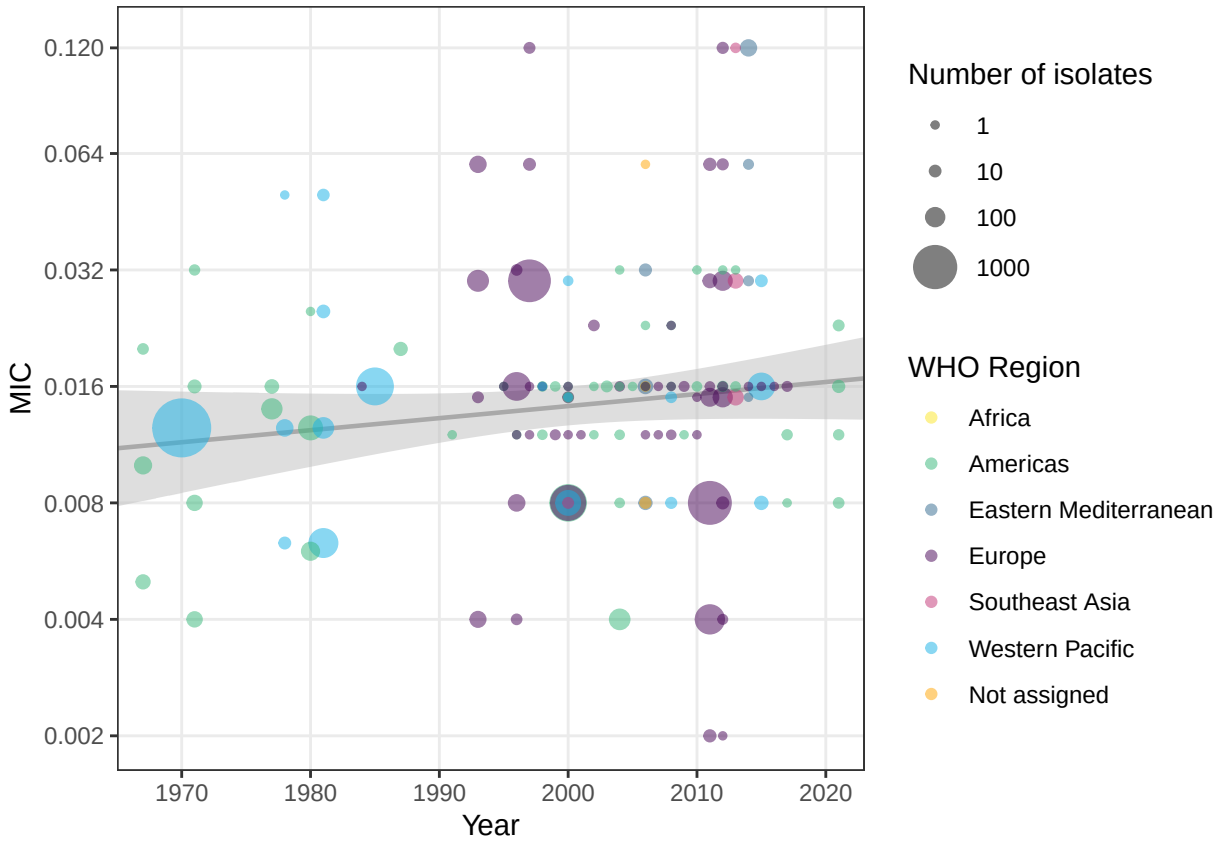
# here comes the model again:
newdata <- data.frame(isolate_yr = seq(X_LOWER, max(dat_nos_only$isolate_yr) + 2))
ci <- predict(mod_weighted_nos_only,
                newdata = newdata,
                interval = "confidence") %>%
  as.data.frame() %>%
  mutate(lwr = ifelse(lwr < Y_LOWER, Y_LOWER, lwr),
         isolate_yr = seq(X_LOWER, max(dat_nos_only$isolate_yr) + 2))

p1 <- ggplot(dat_nos_only %>%
             filter(!is.na(micnos_1)) %>%
             group_by(isolate_yr, micnos_1, who_name) %>%
             summarise(count = sum(count)) %>%
             mutate(who_name = factor(who_name, levels = c("Africa",
                                                           "Americas",
                                                           "Eastern Mediterranean",
                                                           "Europe",
                                                           "Southeast Asia",
                                                           "Western Pacific",
                                                           "Not assigned"))) %>%
         arrange(desc(count))) +
  geom_ribbon(data = ci, aes(ymin = 2**lwr, ymax = 2**upr, x = isolate_yr),
              alpha = 0.5, fill = "grey", colour = NA) +
  geom_line(data = ci, aes(x = isolate_yr, y = 2**fit),
            colour = "darkgrey", linewidth = 0.8) +
  geom_point(aes(x = isolate_yr, y = micnos_1,
                 col = who_name,
                 size = count), alpha = 0.5,
             show.legend = TRUE) +
  scale_color_manual(name = "WHO Region", values = pal, drop = FALSE) +
  scale_size_continuous(range = c(1,10), name = "Number of isolates",
                        breaks = c(1, 10, 100, 1000), limits = SIZE_LIMS) +
  scale_y_continuous(breaks = yticks, minor_breaks = NULL, name = "MIC") +
  scale_x_continuous(limits = c(X_LOWER, 2023), expand = c(0,0),
                     breaks = seq(1970, 2020, 10), minor_breaks = NULL) +
  labs(x = "Year", ylab = "MIC") +
  coord_trans(y = "log2", ylim = YLIM) +
  theme_bw()
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by isolate_yr, micnos_1, and who_name.
## i Output is grouped by isolate_yr and micnos_1.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(isolate_yr, micnos_1, who_name))` for per-operation
##   grouping (`?dplyr::dplyr_by`) instead.
```

```
p1
```

```
## Ignoring unknown labels:
## * ylab : "MIC"
```



All studies: MIC90

```
X_LOWER <- 1927

# here comes the model again:
newdata <- data.frame(isolate_yr = seq(X_LOWER, max(dat_agg$isolate_yr) + 2))
ci <- predict(mod_weighted_agg,
                newdata = newdata,
                interval = "confidence") %>%
  as.data.frame() %>%
  mutate(lwr = ifelse(lwr < Y_LOWER, Y_LOWER, lwr),
         isolate_yr = seq(X_LOWER, max(dat_agg$isolate_yr) + 2))

p2 <- ggplot(dat_agg %>%
             filter(!is.na(mic_90_agg)) %>%
             group_by(isolate_yr, mic_90_agg, who_name) %>%
             summarise(count = sum(count)) %>%
             mutate(who_name = factor(who_name, levels = c("Africa",
                                                           "Americas",
                                                           "Eastern Mediterranean",
                                                           "Europe",
                                                           "Southeast Asia",
                                                           "Western Pacific",
                                                           "Not assigned"))) %>%
         arrange(desc(count))) +
  geom_ribbon(data = ci, aes(ymin = 2**lwr, ymax = 2**upr, x = isolate_yr),
```

```
              alpha = 0.5, fill = "grey", colour = NA) +
  geom_line(data = ci, aes(x = isolate_yr, y = 2**fit),
            colour = "darkgrey", linewidth = 0.8) +
  geom_point(aes(x = isolate_yr, y = mic_90_agg,
                 col = who_name,
                 size = count), alpha = 0.5) +
  scale_color_discrete(name = "WHO Region",
                       type = pal,
                       guide = guide_legend(override.aes = list(size = 5),
                                            ncol = 2),
                       drop = FALSE) +
  scale_size_continuous(range = c(1,10), name = "Number of isolates",
                        breaks = c(1, 10, 100, 1000), limits = SIZE_LIMS) +
  scale_y_continuous(breaks = yticks, minor_breaks = NULL) +
  scale_x_continuous(limits = c(X_LOWER, 2023), expand = c(0,0),
                     breaks = seq(1930, 2020, 10), minor_breaks = NULL) +
  xlab("Year") +
  ylab("MIC90") +
  coord_trans(y = "log2", ylim = YLIM) +
  theme_bw() +
  theme(
    legend.box = "horizontal",
    legend.box.just = "top",
    plot.background = element_rect(fill = "transparent", colour = "transparent")
  )
```

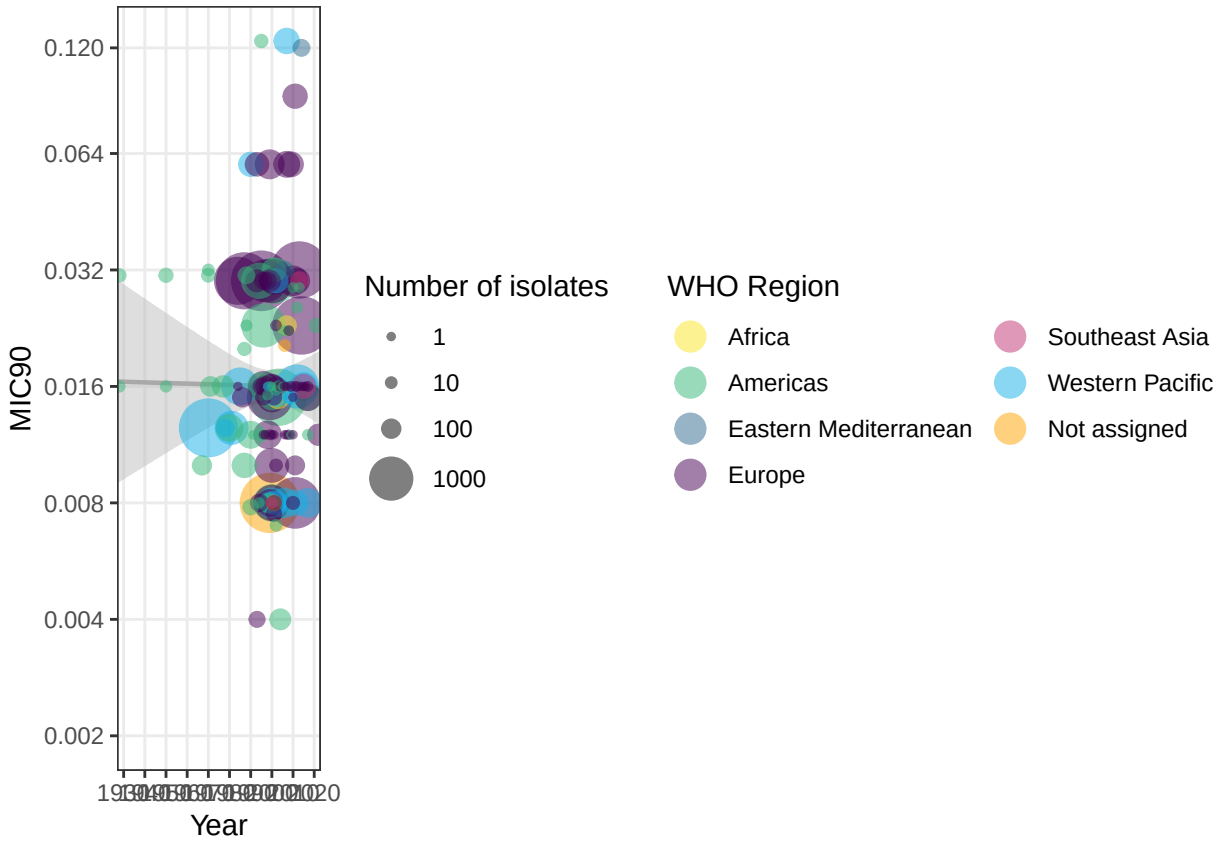
```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by isolate_yr, mic_90_agg, and who_name.
## i Output is grouped by isolate_yr and mic_90_agg.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(isolate_yr, mic_90_agg, who_name))` for per-operation
##   grouping (`?dplyr::dplyr_by`) instead.
```

```
p2
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_ribbon()`).
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_line()`).
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_point()`).
```



Putting it all together:

```
p <- plot_grid(p2 + theme(legend.position = "none",
                          plot.margin = margin(t = 20, r = 0, b = 0, l = 5, unit = "pt")),
               p1 + theme(legend.position = "none",
                          plot.margin = margin(t = 20, r = 0, b = 0, l = 5, unit = "pt")),
               labels = c("(a) MIC90: all studies",
                          "(b) Studies documenting MIC for all isolates"),
               label_fontface = "plain",
               rel_widths = c(0.6, 0.4),
               label_x = c(0.15, 0.1), hjust = 0, label_size = 12)
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_ribbon()`).
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_line()`).
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_point()`).
```

```
## Ignoring unknown labels:
## * ylab : "MIC"
```

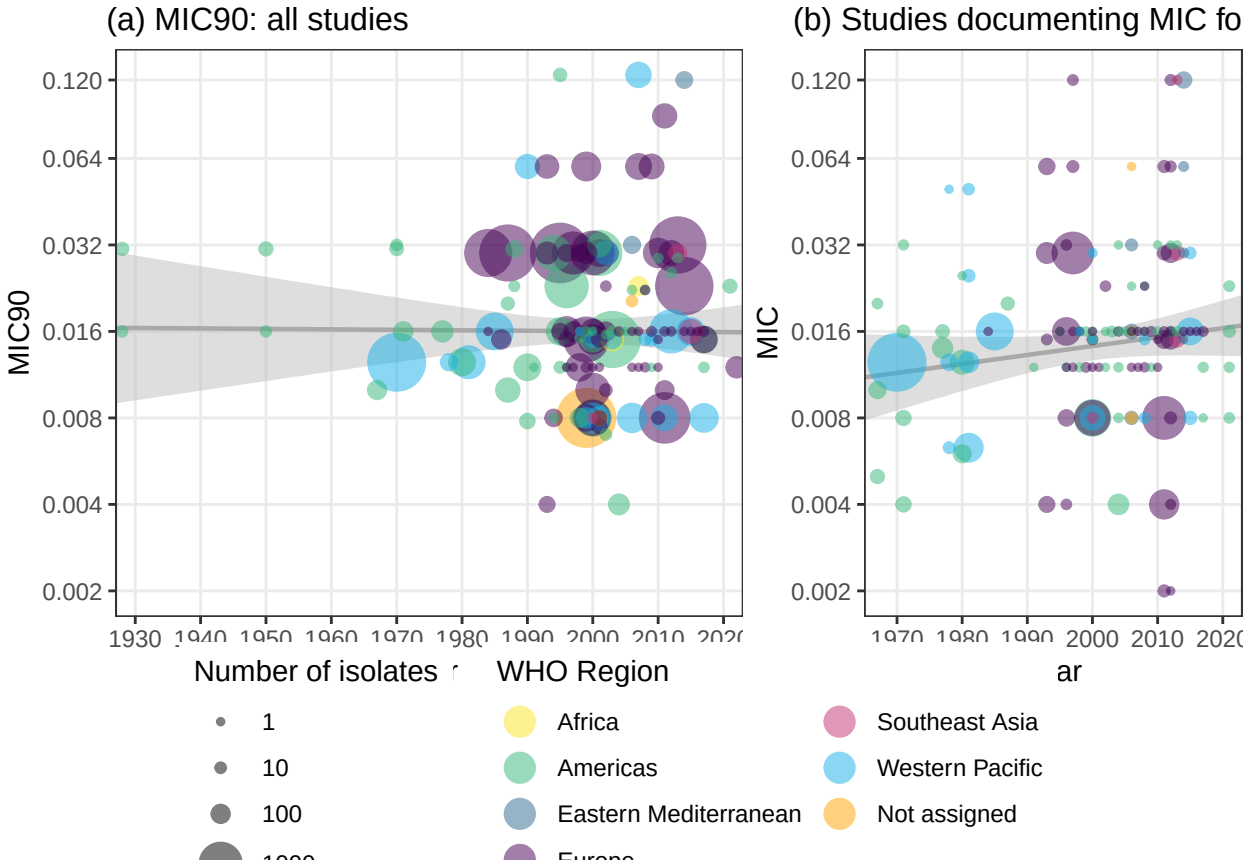
```
plot_grid(p, get_legend(p2), ncol = 1, rel_heights = c(0.8, 0.2))
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_ribbon()`).
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_line()`).
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_point()`).
```

```
## Warning in get_plot_component(plot, "guide-box"): Multiple components found;
## returning the first one. To return all, use `return_all = TRUE`.
```




```
ggsave("figs/MIC_regression_twice.png", height = 7, width = 9, bg = "transparent")
ggsave("figs/MIC_regression_twice.svg", height = 7, width = 9, bg = "transparent")
```

LH coming back, making times match vertically.

```
p <- plot_grid(
          p1 + theme(legend.position = c(-0.5, 0.5),
                     plot.margin = margin(t = 20, r = 0, b = 0, l = 215,
                                          unit = "pt")),
          p2 + theme(legend.position = "none",
                     plot.margin = margin(t = 20, r = 0, b = 0, l = 5,
                                          unit = "pt")),
          labels = c("(a) Trend based on individual MICs",
                     "(b) Trend based on summarised MICs"),
          label_fontface = "plain",
          nrow = 2,
          #label_x = c(0.07, 0.45),
          label_x = c(0.45, 0.07),
          hjust = 0, label_size = 12)
```

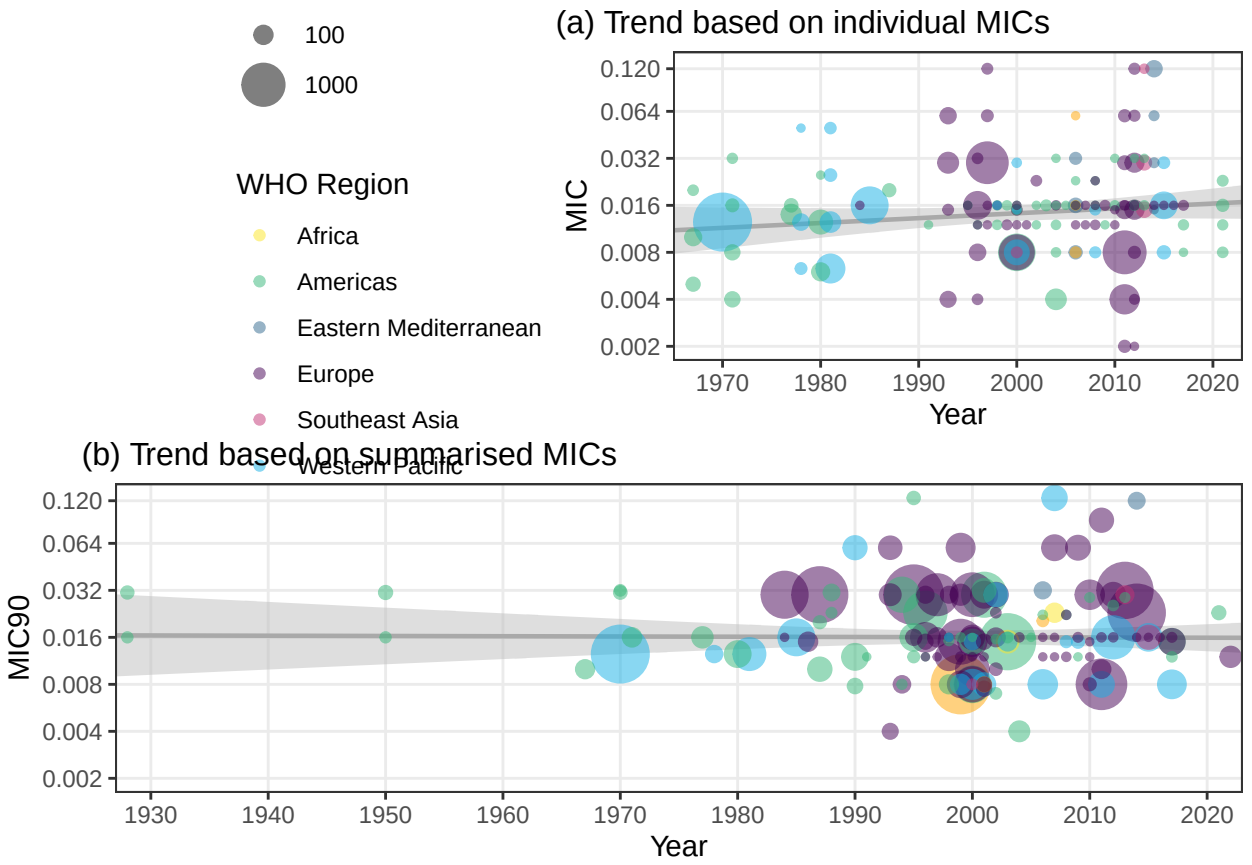
```
## Ignoring unknown labels:
## * ylab : "MIC"
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_ribbon()`).

## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_line()`).

## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_point()`).
```

```
p
```




```
ggsave("figs/MIC_regression_twice_vertical.png", height = 9, width = 8, bg = "white")
ggsave("figs/MIC_regression_twice_vertical.svg", height = 9, width = 8, bg = "white")
```

TODO - report Rsq

Total isolates by MIC90/raw MIC

Figure 3: what are the spreads of the two subsets?

```
sum(dat_raw$count)
```

```
## [1] 40630
```

```
hist_dat <- dat_raw %>% # from Rhys' aggregate file
  filter(!is.na(mic90_1)) %>%
  bind_rows(dat_nos_only) %>% # NOS only
  pivot_longer(cols = c(micnos_1, mic90_1), names_to = "type") %>%
  filter(!is.na(value)) %>%
  mutate(value = round(value, digits = 4)) %>%
```

```r
  summarise(count = sum(count), .by = c(value, type)) %>%
  mutate(logmic = log2(value)) %>%
  mutate(type = ifelse(type == "mic90_1",
                       "Only summarised MIC reported (MIC90)",
                       "Individual MIC reported"))

hist_dat %>%
  summarise(.by = type, count = sum(count))
```

```
## # A tibble: 2 x 2
##   type                                  count
##   <chr>                                 <int>
## 1 Only summarised MIC reported (MIC90) 31229
## 2 Individual MIC reported               9401
```

```r
x_breaks = c(0.002, 0.004, 0.008, 0.016, 0.032, 0.064, 0.12)

# for labels on bars:
to_lab <- hist_dat %>%
  # only highest ones:
  filter((type == "Only summarised MIC reported (MIC90)" & count > 2000) |
           (type == "Individual MIC reported" & count > 1000)) %>%
  mutate(nudge_yy = ifelse(type == "Individual MIC reported",
                           1200, 300),
         hjustt = ifelse(count %in% c(2727, 2100, 3097), 0, 0.5),
         vjustt = ifelse(count %in% c(2727, 3097, 2030), 0.5, 1),
         #nudge_x = ifelse(count %in% c(2727, 2100), 0, 0.5),
         lab = ifelse(count %in% c(2727, 3097, 2030),
                      paste0("MIC:\n", value, "\n(", count, ")"),
                      paste0("MIC: ", value, "\n(", count, ")")))

xlim = c(0.0019, 0.15)

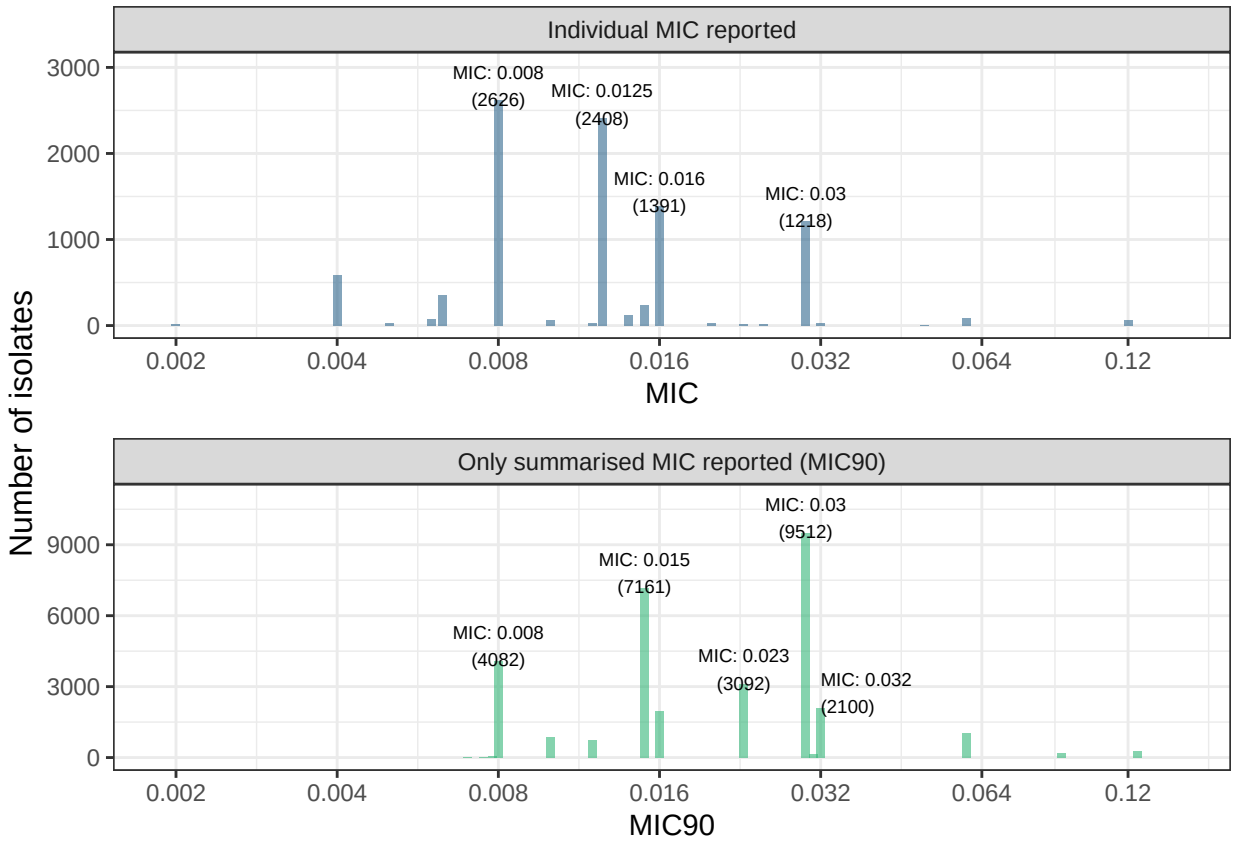
p1 <- ggplot(data = hist_dat %>%
               filter(type == "Only summarised MIC reported (MIC90)")) +
  geom_bar(aes(x = logmic, y = count,
           fill = type),
           alpha = 0.6,
           stat = "identity",
           width = 0.05) +
  # here are labels on top of higher bars:
  geom_text(aes(x = logmic, y = count, label = lab,
                hjust = hjustt),
            data = to_lab %>%
              filter(type == "Only summarised MIC reported (MIC90)"),
             nudge_y = 1500,
            vjust = 1,
            size = 2.5) +
  facet_wrap(~ type, ncol = 1, scales = "free_y") +
  scale_x_continuous(breaks = log2(x_breaks),
                     labels = x_breaks,
                     limits = log2(xlim)) +
  scale_fill_manual(values = pal[2]) +
  ylab("") +
```

```
  xlab("MIC90") +
  theme_bw() +
  theme(legend.position = "none")

p2 <- ggplot(data = hist_dat %>%
               filter(type == "Individual MIC reported")) +
  geom_bar(aes(x = logmic, y = count,
           fill = type),
           alpha = 0.6,
           stat = "identity",
           width = 0.05) +
  # here are labels on top of higher bars:
  geom_text(aes(x = logmic, y = count, label = lab),
            data = to_lab %>%
               filter(type == "Individual MIC reported"),
             nudge_y = 400,
            vjust = 1,
            size = 2.5) +
  facet_wrap(~ type, ncol = 1, scales = "free_y") +
  scale_x_continuous(breaks = log2(x_breaks),
                     labels = x_breaks,
                     limits = log2(xlim)) +
  scale_fill_manual(values = pal[3]) +
  ylab("") +
  xlab("MIC") +
  theme_bw() +
  theme(legend.position = "none")

p <- plot_grid(p2, p1, ncol = 1)

ggdraw(p) +
  draw_label("Number of isolates", x = 0, y = 0.5, angle = 90,
             vjust = 1.5, size = 12)
```




```
ggsave("figs/all_mics.png", height = 6, width = 6)
ggsave("figs/all_mics.svg", height = 6, width = 6)

# ggplot(data = hist_dat) +
#   geom_bar(aes(x = logmic, y = count,
#            fill = type),
#            alpha = 0.6,
#            stat = "identity",
#            width = 0.05) +
#   # here are labels on top of higher bars:
#   geom_text(aes(x = logmic, y = count, label = lab,
#                 hjust = hjustt, vjust = vjustt),
#             data = to_lab,
#              nudge_y = to_lab$nudge_yy,
#             size = 2.5) +
#   # abandoning
#   facet_wrap(~ type, ncol = 1, scales = "free_y") +
#   scale_x_continuous(breaks = log2(x_breaks),
#                      labels = x_breaks,
#                      limits = log2(xlim)) +
#   scale_fill_manual(values = pal[2:3]) +
#   ylab("") +
#   xlab("MIC90") +
#   theme_bw() +
#   theme(legend.position = "none")
```

Ordinal regression

What happens when we try to reapply the linear regression treating MICs as ordinal? We could really treat MICs as categories as intermediate MICs are not (usually) tested. For this reason, we've had a look at ordinal regression in the chunk below. We resolved not to do this because the data was all created under different settings with (probably) different dilutions, different methods of calculating MIC90, etc.

First let's have a look at raw MICs only:

```
library(MASS)
```

```
##
## Attaching package: 'MASS'
```

```
## The following object is masked from 'package:dplyr':
##
##     select
```

```
ordinal_regn_please <- function(dat, bins, ylab,
                                weight = TRUE,
                                xlower = 1927,
                                xupper = 2024){
  dat <- mutate(dat,
                mic_binned = cut(mic,
                                 breaks = c(0, bins),
                                 ordered_result = TRUE))

  # two poss models:
  if (weight){
    model <- polr(mic_binned ~ isolate_yr,
                  data = dat,
                  Hess = TRUE,
                  weights = count, # here, weights are frequencies
                  method = "probit") # fails to converge under logit: fatter tail
  } else {
    message("here")
    model <- polr(mic_binned ~ isolate_yr,
                  data = dat %>% uncount(count),
                  Hess = TRUE)
  }

  # Fit the model - no need to log: that would be for visualisation purposes only under ordinal model
  # tried using wts from logged model ......... not thinking too hard about that but we should at some
  # it looks like weights need to be integers as we end up assuming binomial glm for ordinal *logistic*
  # haven't thought too hard about this ... may need to be probit link

  print(summary(model))
  # note the slope here

  new_data = data.frame(isolate_yr = seq(xlower + 2,
                                         xupper - 2))
  preds <- predict(model,
                   newdata = new_data,
                   type = "probs") %>%
    as.data.frame() %>%
    mutate(max_probs = max.col(across(starts_with("(")))) %>%
```

```
  mutate(isolate_yr = new_data$isolate_yr)

if (length(unique(preds$max_probs)) == 1){
  message(paste0("Most probable class stays the same:",
                 unique(preds$max_probs)))
} else {
  message(paste0(c("Most probable class changes:\n", unique(preds$max_probs)),
                   collapse = " "))
}


preds_long <- pivot_longer(preds, starts_with("(")) %>%
  mutate(lower = str_extract(name, "(?<=\\()[^,]+") %>% as.numeric(),
         upper = str_extract(name, "(?<=,)[^]]+") %>% as.numeric(),
         height = lower + (upper - lower)*value) %>%
  filter(lower != 0)

preds_change <- preds %>%
  group_by(max_probs) %>%
  summarise(xmin = first(isolate_yr) - 0.5,
            xmax = last(isolate_yr) + 0.5) %>%
  mutate(ymin = bins[max_probs - 1],
         ymax = bins[max_probs])

# so now we have colour bars representing class probability between the lower
# and upper extents of the bins ... which don't change much because there isn't
# much change in MIC slope over time
ggplot(data = dat) +
  geom_point(aes(x = isolate_yr, y = mic, size = count),
             alpha = 0.5, col = "grey30") +
  scale_y_continuous(trans = "log2", breaks = bins, minor_breaks = NULL) +
  geom_ribbon(data = preds_long,
              aes(ymin = lower, ymax = height, fill = name, x = isolate_yr),
              alpha = 0.5) +
  scale_fill_discrete("MIC interval") +
  scale_size_continuous(range = c(1,10), name = "Number of\nisolates",
                        breaks = c(1, 10, 100, 1000), limits = SIZE_LIMS) +
  # (the -1 is because I removed the bottom bin)
  # this is a bit gross but I've put the line in the middle
  # geom_hline(yintercept = 2**(sum(log2(bins[preds$max_probs[1] + c(0,-1)])) / 2)) +
  # annotate(geom = "rect",
  #          xmin = xlower,
  #           xmax = max(new_data$isolate_yr),
  #           ymin = bins[preds$max_probs[1] - 1],
  #           ymax = bins[preds$max_probs[1]], col = "black", fill = NA) +
  geom_rect(data = preds_change,
            aes(xmin = xmin, xmax = xmax, ymin = ymin, ymax = ymax),
            col = "black", fill = NA) +
  scale_x_continuous(breaks = seq(ceiling(xlower / 10) * 10,
                                  floor(xupper / 10) * 10, 10),
                     minor_breaks = NULL, expand = FALSE,
                     limits = c(xlower - 0.1, xupper + 0.1)) +
  xlab("Year") +
```

```
    ylab(ylab) +
    theme_bw() +
    theme(
      legend.box = "horizontal",
      legend.box.just = "top",
      plot.background = element_rect(fill = "transparent", colour = "transparent"),
      plot.title = element_text(size=11)
    )
}

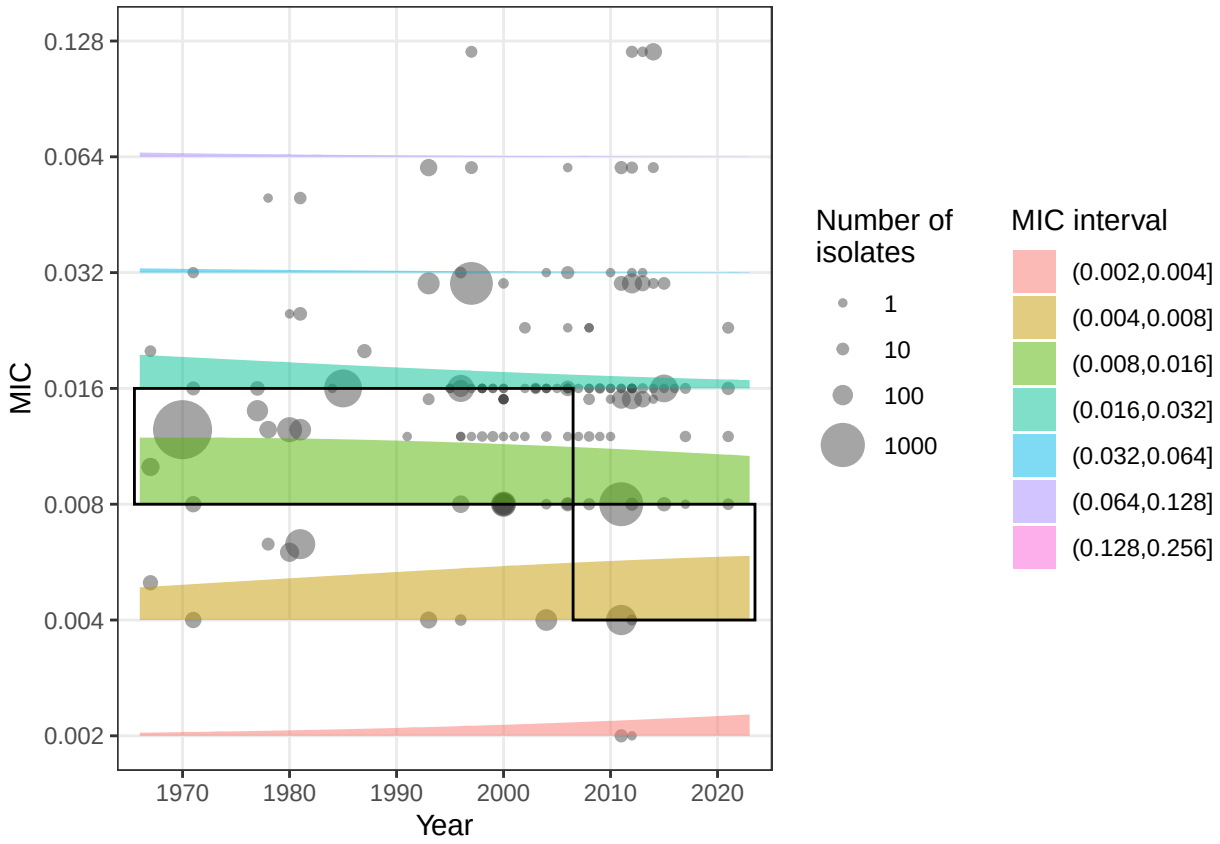
bins = c(0.002, 0.004, 0.008, 0.016, 0.032, 0.064, 0.128, 0.256)
# bins = 2**(log2(bins) - 0.5)

# that's interesting .. this slips down a class!
p1 <- ordinal_regn_please(dat_nos_only %>%
                            mutate(mic = micnos_1),
                          # bins = c(0.002, 0.004, 0.008, 0.016, 0.032, 0.064, 0.128),
                          weight = TRUE,
                          bins = bins, xupper = 2025,
                          xlower = 1964, ylab = "MIC")
```

```
## Call:
## polr(formula = mic_binned ~ isolate_yr, data = dat, weights = count,
##     Hess = TRUE, method = "probit")
##
## Coefficients:
##               Value Std. Error t value
## isolate_yr -0.01754  1.082e-05   -1620
##
## Intercepts:
##                              Value         Std. Error    t value
## (0,0.002]|(0.002,0.004]     -3.806530e+01  4.000000e-04 -9.224765e+04
## (0.002,0.004]|(0.004,0.008] -3.655150e+01  2.000000e-03 -1.818789e+04
## (0.004,0.008]|(0.008,0.016] -3.519760e+01  2.240000e-02 -1.568382e+03
## (0.008,0.016]|(0.016,0.032] -3.387900e+01  2.570000e-02 -1.318738e+03
## (0.016,0.032]|(0.032,0.064] -3.284970e+01  3.570000e-02 -9.196834e+02
## (0.032,0.064]|(0.064,0.128] -3.253310e+01  3.590000e-02 -9.068079e+02
## (0.064,0.128]|(0.128,0.256]  2.068948e+12  3.590000e-02  5.766852e+13
##
## Residual Deviance: 23170.82
## AIC: 23186.82
```

```
## Most probable class changes:
##  4 3
```

```
p1
```

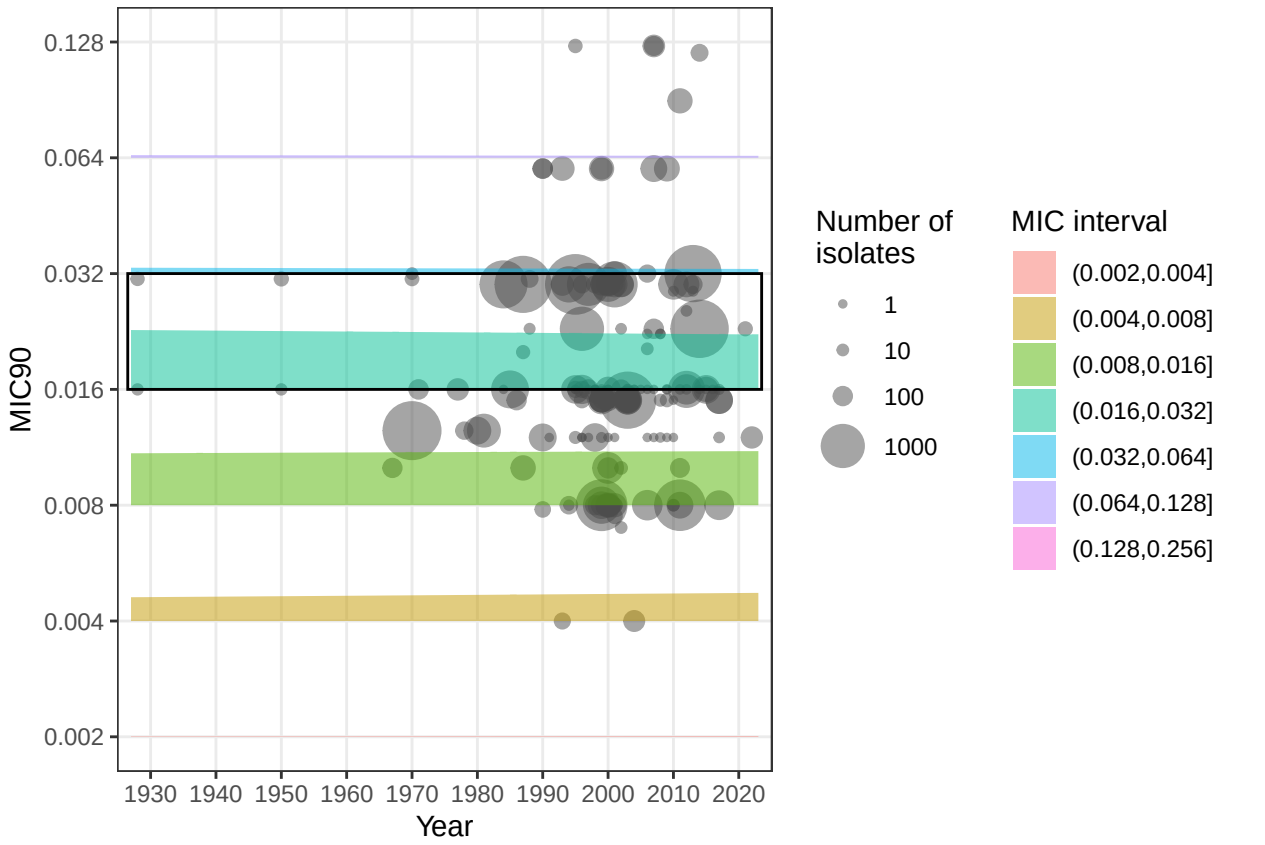



```
p2 <- ordinal_regn_please(dat_agg %>% mutate(mic = mic_90_agg),
                          bins = bins,
                          xlower = 1925, xupper = 2025,
                          ylab = "MIC90")
```

```
## Call:
## polr(formula = mic_binned ~ isolate_yr, data = dat, weights = count,
##     Hess = TRUE, method = "probit")
##
## Coefficients:
##                Value Std. Error t value
## isolate_yr -0.001305  1.298e-05  -100.5
##
## Intercepts:
##                              Value       Std. Error  t value
## (0,0.002]|(0.002,0.004]          -7.2184      0.0007 -10619.5774
## (0.002,0.004]|(0.004,0.008]      -5.2314      0.0034  -1543.1434
## (0.004,0.008]|(0.008,0.016]      -3.5253      0.0262   -134.7889
## (0.008,0.016]|(0.016,0.032]      -2.4632      0.0264    -93.4417
## (0.016,0.032]|(0.032,0.064]      -0.8936      0.0287    -31.1745
## (0.032,0.064]|(0.064,0.128]      -0.3850      0.0288    -13.3834
## (0.064,0.128]|(0.128,0.256]      29.7652      0.0288   1034.6071
##
## Residual Deviance: 99561.28
## AIC: 99577.28
```

```
## Most probable class stays the same:5
```

```
p2
```



Caption: Ordinal regression of MIC data (as in Figure 4): coloured regions represent predicted probability of MIC interval over time; the most probable interval, which does not change over time in either case, is highlighted with a black rectangle. Intervals are right-inclusive: the interval (0.008, 0.016] includes MIC 0.016 and excludes MIC 0.008.

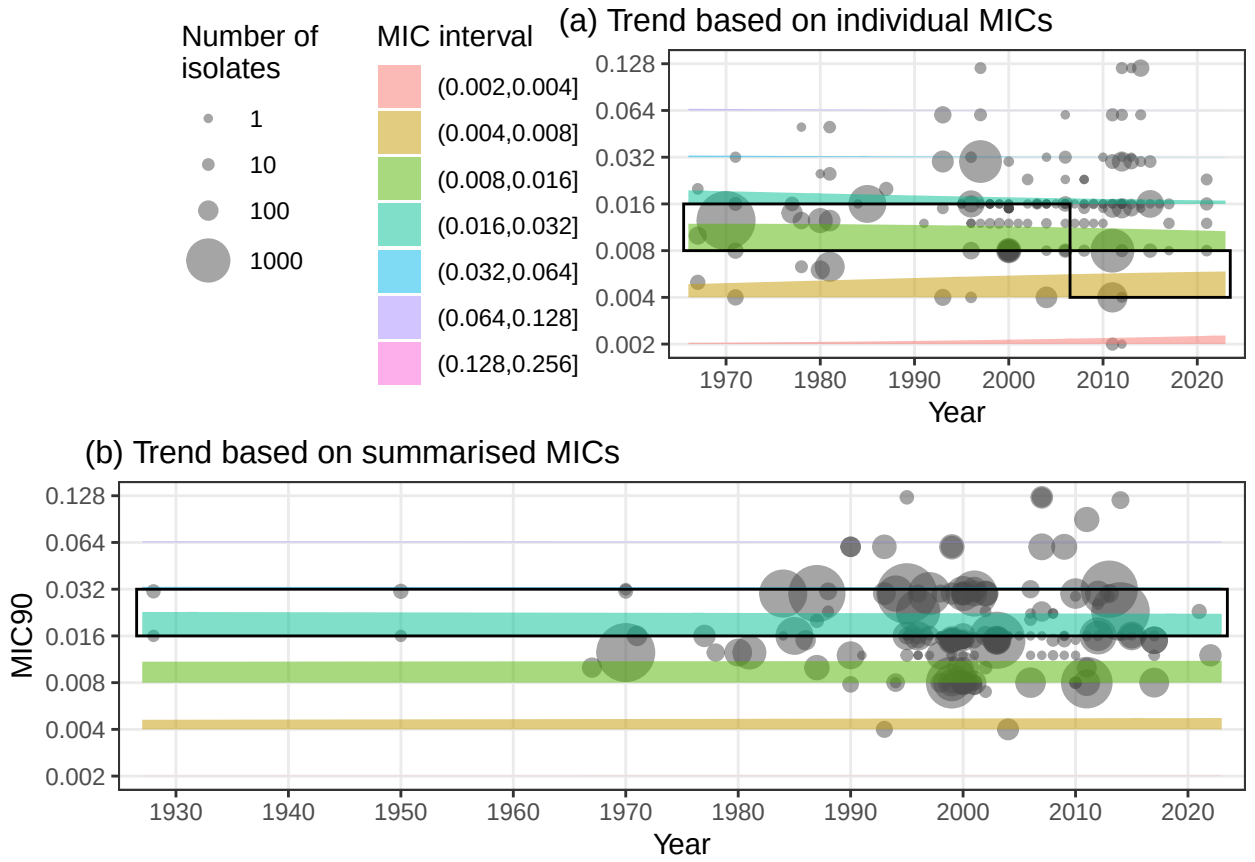
Here we put the subplots together:

```
# p <- plot_grid(p2 + theme(legend.position = "none"),
#           p1 + theme(legend.position = "none"),
#           rel_widths = c(0.6, 0.45))
#
# plot_grid(p, get_legend(p2), ncol = 1, rel_heights = c(0.8, 0.25))
#
# ggsave("figs/ordinal_regression.png", height = 7, width = 9, bg = "transparent")
# ggsave("figs/ordinal_regression.svg", height = 7, width = 9, bg = "transparent")


p <- plot_grid(
          p1 + theme(legend.position = c(-0.5, 0.5),
                     plot.margin = margin(t = 20, r = 0, b = 0, l = 212,
                                          unit = "pt")),
          p2 + theme(legend.position = "none",
                     plot.margin = margin(t = 20, r = 0, b = 0, l = 5,
                                          unit = "pt")),
          labels = c("(a) Trend based on individual MICs",
                     "(b) Trend based on summarised MICs"),
```

```
          label_fontface = "plain",
          nrow = 2,
          #label_x = c(0.07, 0.45),
          label_x = c(0.45, 0.07),
          hjust = 0, label_size = 12)

p
```




```
ggsave("figs/ordinal_regression.png", height = 9, width = 8, bg = "white")
ggsave("figs/ordinal_regression.svg", height = 9, width = 8, bg = "white")
```